# UVM Introduction 1

## Electrical and Information Technology, LTH  Lund University

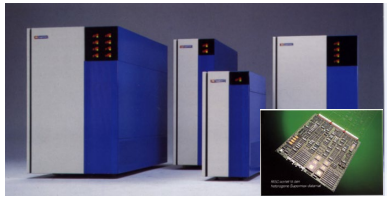
2026-09-21

**Ole Kristoffersen,**
**Senior Specialist Emerging Verification Technologies at Ericsson**

# My journey

## 40 years HW/ASIC verification experiences

- Implemented testing for HW prototype and production testing
  - Test of CPU, memory and IO circuit boards
  - Also creating to check peripheral devices (disks, back-up systems etc.)
- IP block and subsystem verification for radio uplink/downlink
- Deployed C-based and UVM-based verification systems in ASIC organizations
- From 2021, responsible ASIC/FPGA verification strategy, implementation of verification methodology improvements and technology scouting



Dansk Data Electronik (12 years)
Multi Processor Unix Server



Olicom/Intel (4 years)
LAN switch/router products



Nokia/Renesas (11 years)
Mobil ASIC



Ericsson (8 years)
Radio ASIC



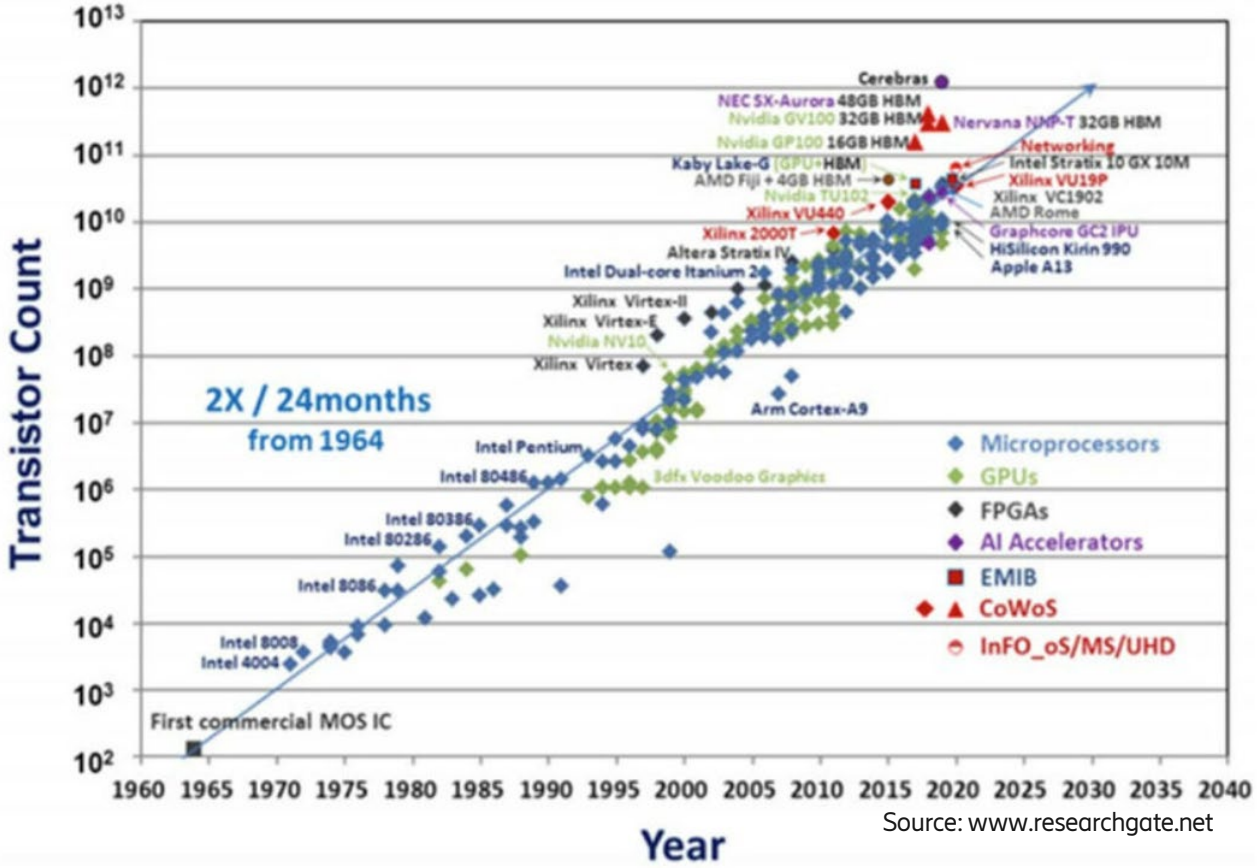

Ericsson (5 year)
ASIC/FPGA verification methodology
and technology scouting

# Topics

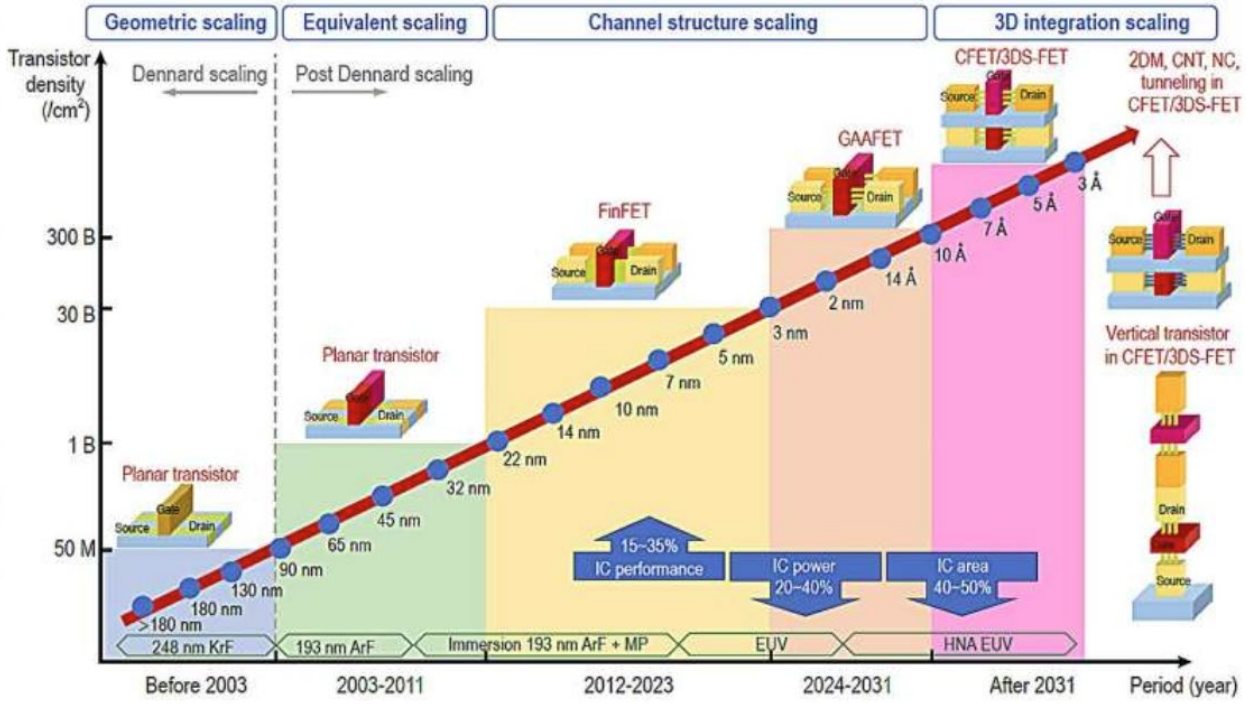
- Verification Challengers
- TB Maintains Challenges
- Introduction to UVM
- Coffee Break
- UVM Concept
- UVM TB Reusability and Scalability
- UVM Key Takeaways
- How AI Impacts the Way We Work
- Summary


Verification Challengers

# Verification Challengers – ASIC Transistor Count Trends

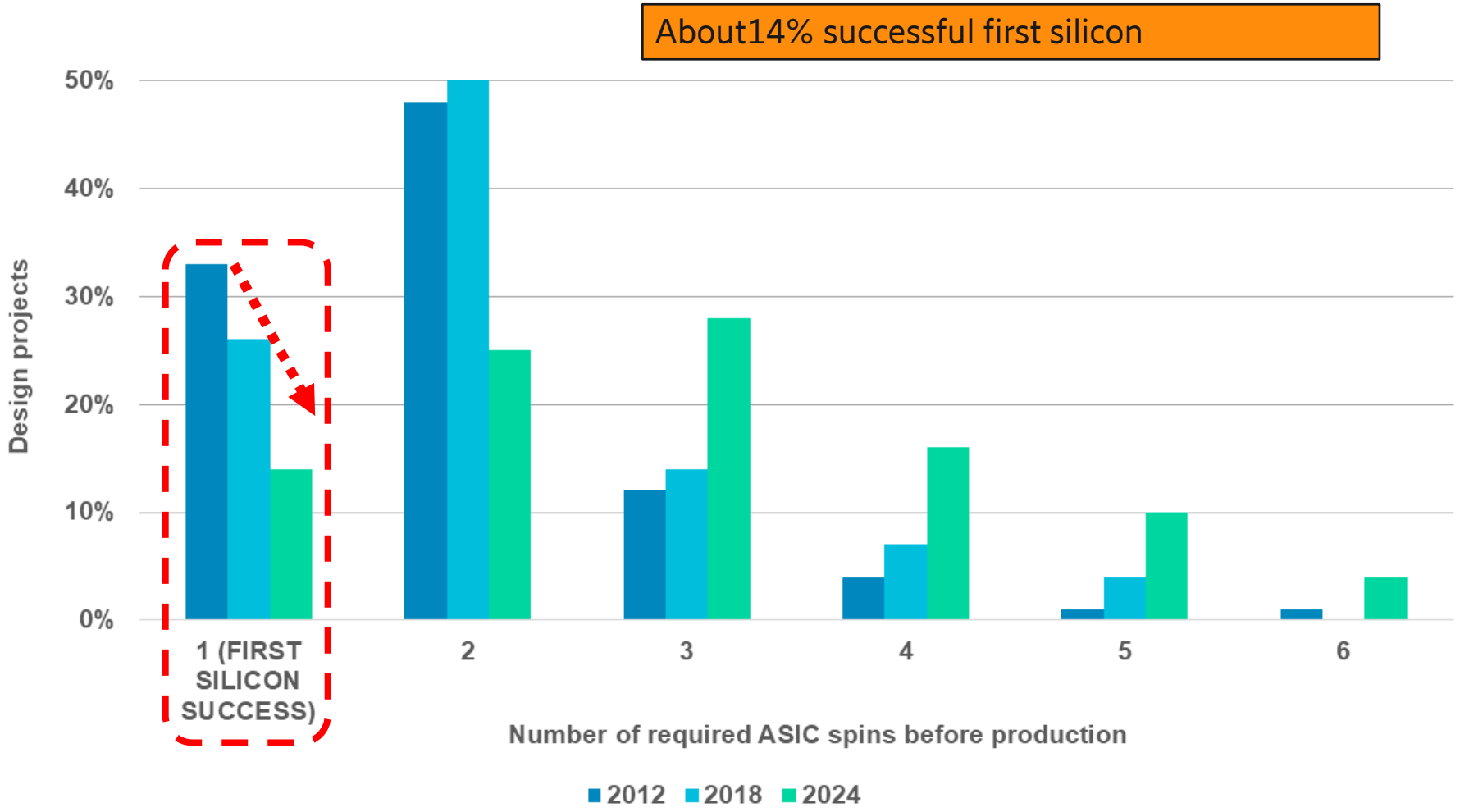



Source: www.researchgate.net



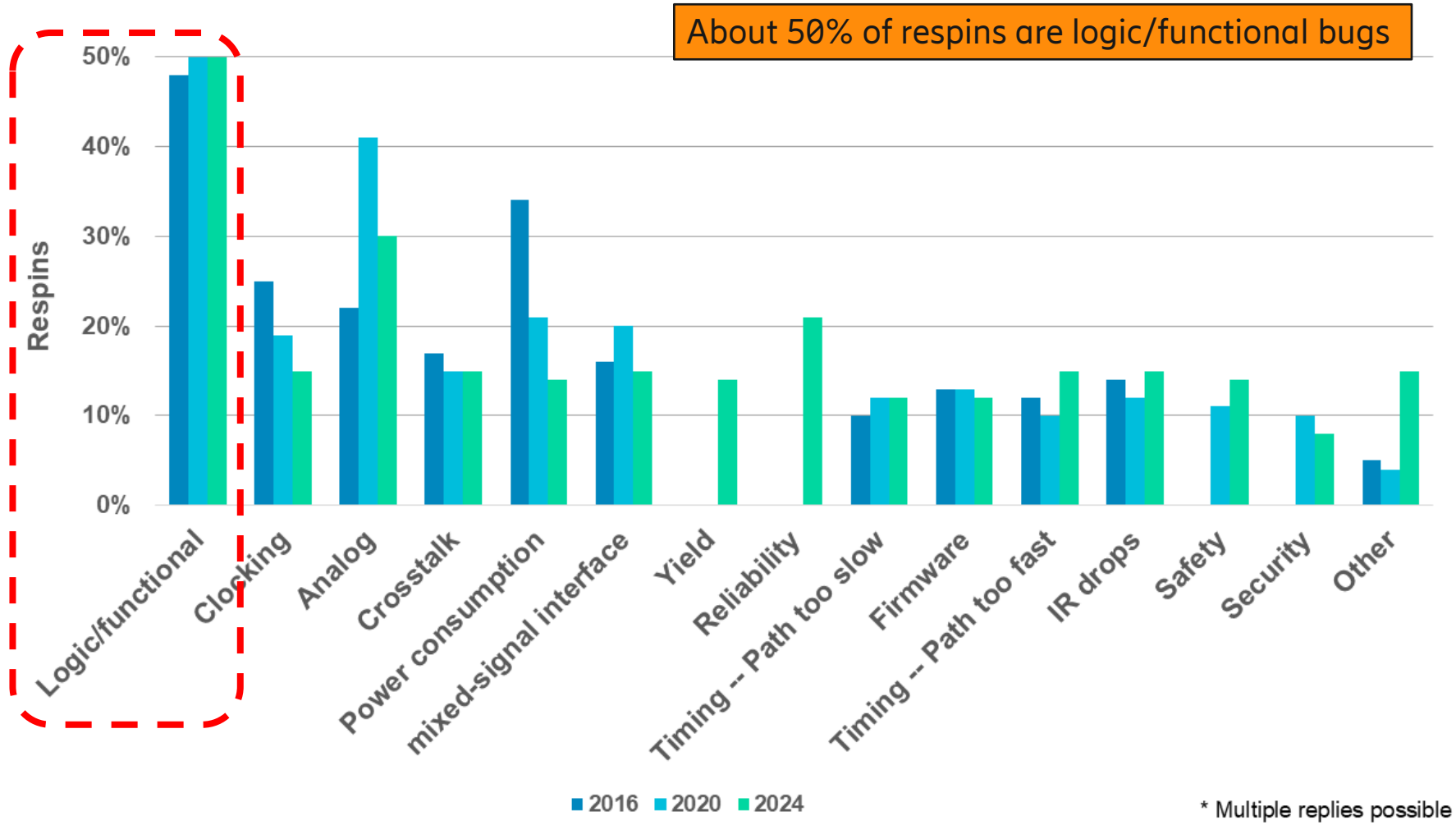

Source: www.techxplore.com

# Verification Challengers – ASIC Re-spins




Source: Wilson Research Group and Siemens EDA

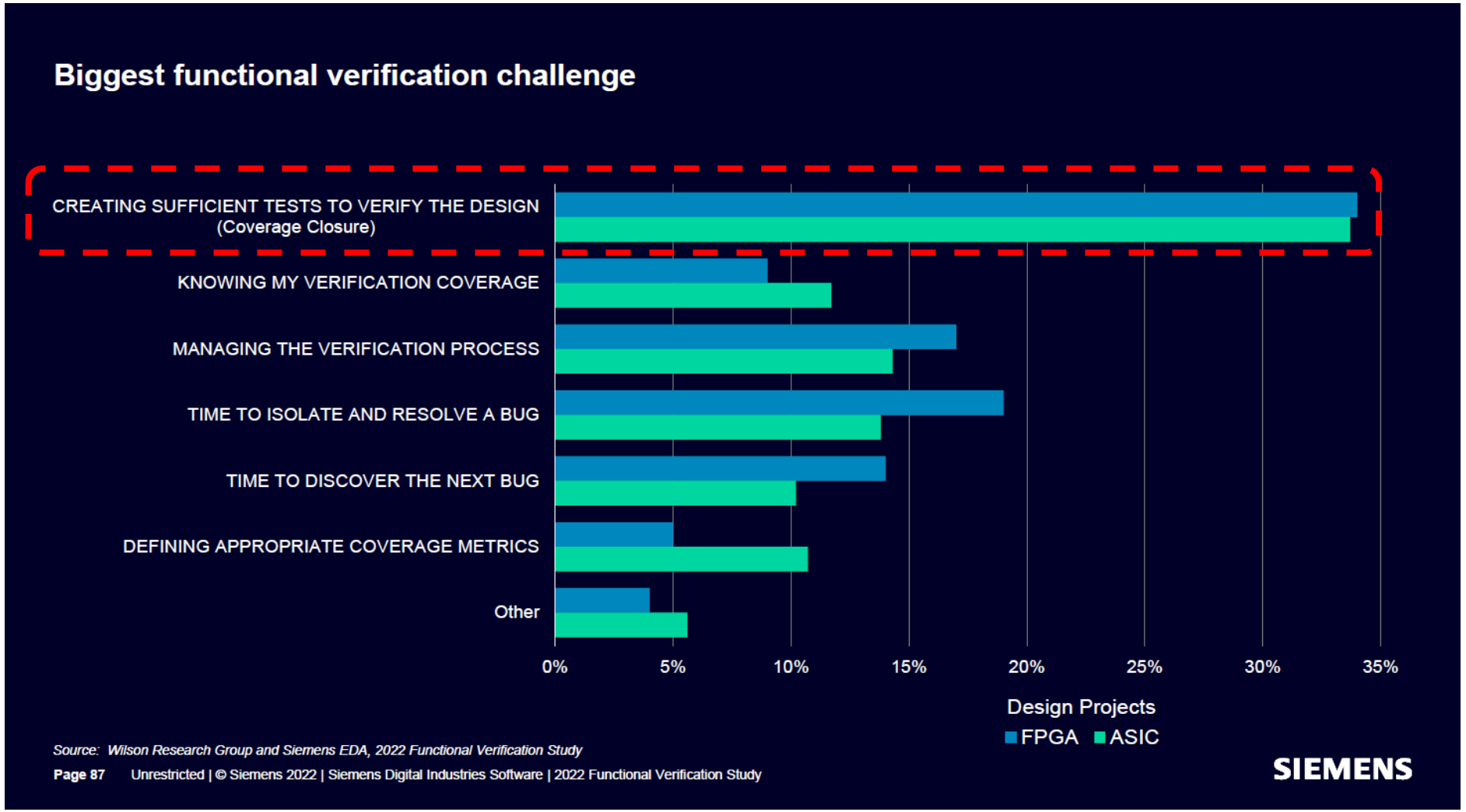
# Verification Challengers – Root Causes of Bugs




* Multiple replies possible

Source: Wilson Research Group and Siemens EDA

# Verification Challengers – Verification Sign-off Challenge



# Verification Challengers – Summary

**The ASIC design becomes more complex**

- The number of transistors increases significantly with each generation

**As designs become more complex, the challenges for the verification environment increase**

- The test benches must be well-structured, flexible, reusable and scalable

**Achieving success with first silicon is becoming increasingly challenging**

- Complex designs require thorough verification planning to identify and create corner cases

**Finding bugs at a late-stage increases costs may loses time-to-market or requiring an ASIC re-spin**

- The root causes are mostly due to verification quality, which needs to be improved


TB Maintains Challenges

# TB Maintains Challengers – Creation of Simple TB from Scratch

**Typical IP design start with simple design**

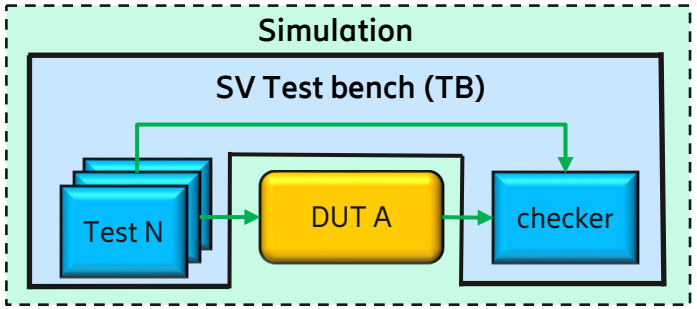
- A simple SV test bench is created to verify the IP design

**The IP design requirements change during an ASIC project**

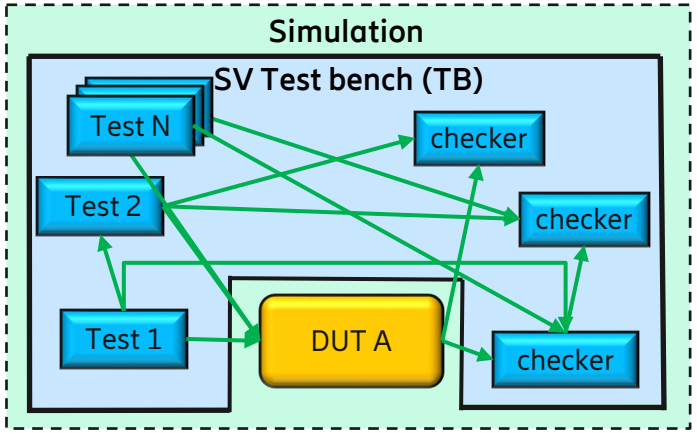
- Testing of new features does not fit very well into an already TB structure

**The IP design grows with new features for the next ASIC project**

- Continue to update the TB to verify new functionality in existing TB
- Realize during the project, the TB becomes more and more complicated and difficult to implement tests



Simple SV TB using direct test



Unstructured SV TB using direct test

High risk of ASIC bugs due to lack of corner cases to verify the IP design

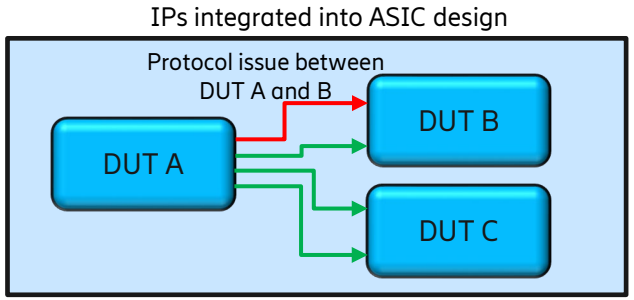
# TB Maintains Challengers – Refactor TB to be Structured and Use Randomization

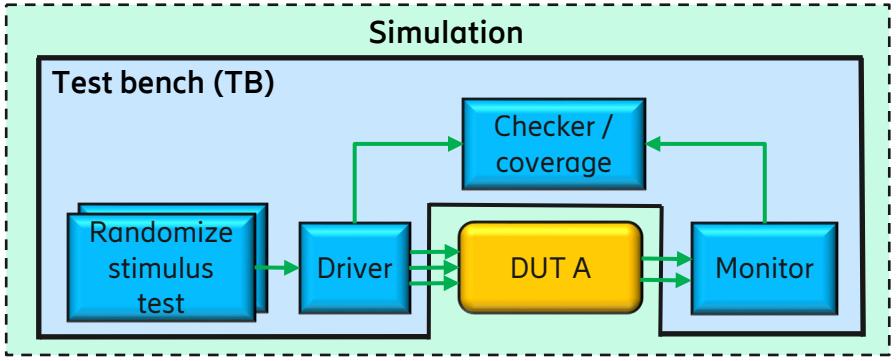
## The third generation of the IP design

- Decided to refactor the TB with driver, monitor and using randomization

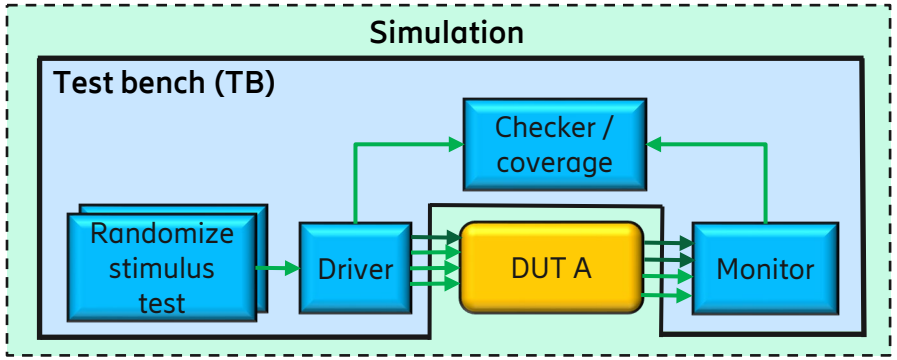
## The IP design changed again in the project with new port signals

- Some part of TB need to be updated in order to be able to test the updated IP design




SV TB with randomized stimulus tests, utilizing a driver and monitor




Updated SV TB with randomized stimulus tests, utilizing a driver and monitor

**High risk of ASIC failure due to top-level integration between different IPs**

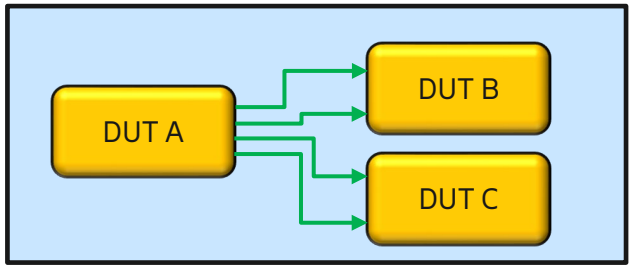
# TB Maintains Challengers – Refactor TB Driver and Monitors

**The fourth generation of the IP design**

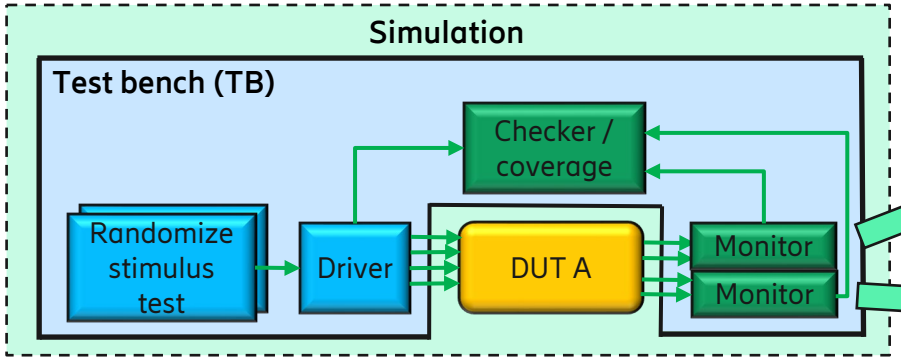
- Refactor the TB to prevent future IP integration protocol bugs

**The same monitor must be shared across different IP TBs**

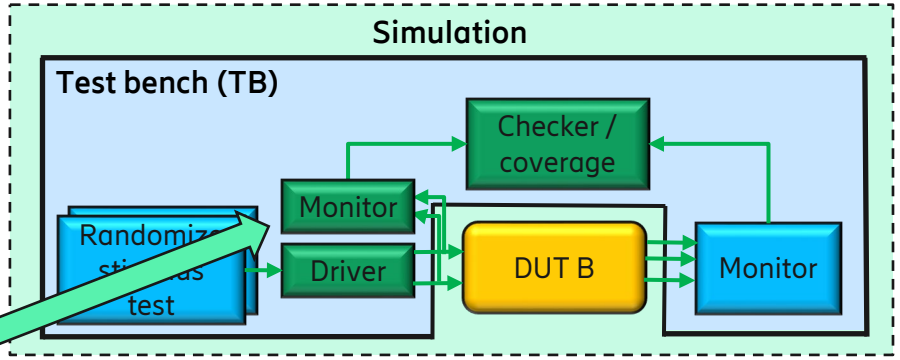
- This ensure both IPs adhere to the same protocol
- The TB structure should be aligned to be able to reuse monitor across the TBs



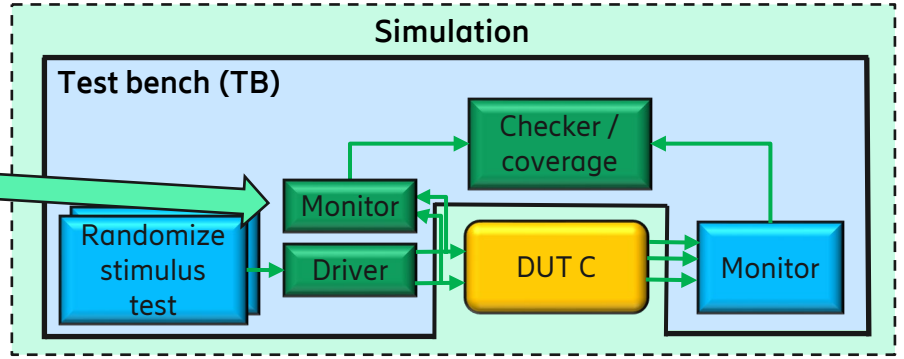

IPs integrated into ASIC design




DUT A TB shares monitors with DUT B and C TBs




DUT B TB shares a monitor with DUT A TB




DUT C TB shares a monitor with DUT A TB

# TB Maintains Challengers – Key Takeaways

**Create a TB architecture that is easy to scale with additional IP functionality and port interfaces**

- Otherwise, you end up with spaghetti code that is not easy to maintain
- This also minimize the risk of escaping design bugs

**Use randomized testing method instead of creating direct tests**

- Easier to maintain and scale as design feature grows
- It's easier to create corner cases you haven't thought about and it can find more design bugs

**The interfaces between IPs must be verified with common drivers and monitors**

- This is necessary to find interface protocol bugs between the IPs
- Requires common TB structure between the different TBs


Introduction to UVM

# Introduction to UVM – What is UVM (Universal Verification Methodology)

**UVM is a standardized TB framework**

- It provides a structured approach for creating reusable, scalable, and robust TBs
- Used for verifying integrated circuit designs

**UVM Library in SystemVerilog**

- The library contains all the base classes and utilities needed to build a TB
- Verification engineers create custom TB classes by extending UVM base classes

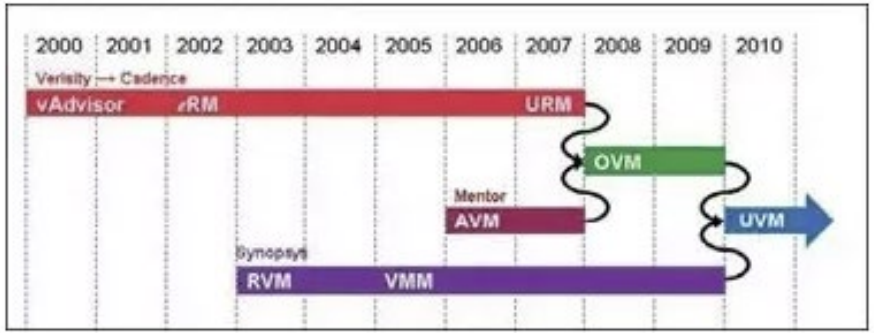
**UVM base classes and utilities provides a framework**

- Creating testbench component builds the TB
- Run phases, with each phase serving a specific function
- Sequences generate stimuli that drive the DUT

# Introduction to UVM – History and Background

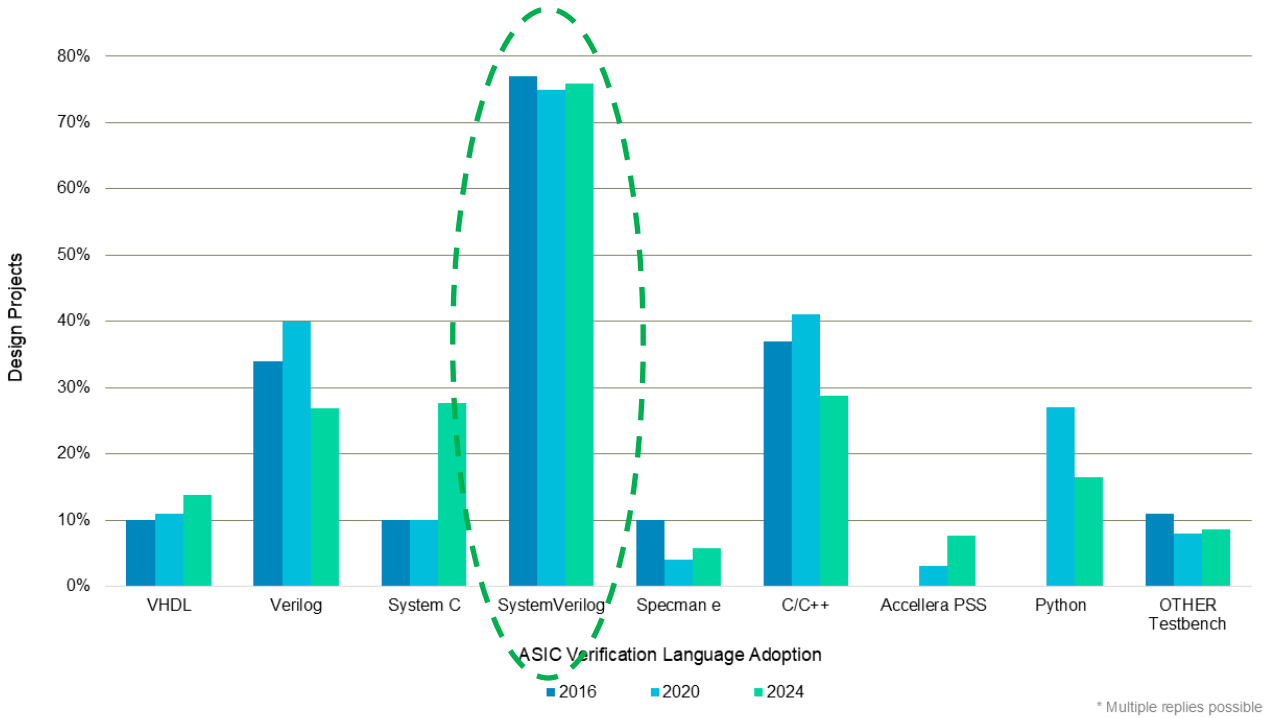
## UVM Timeline

- **2009**: Accellera forms the UVM Working Group
- **2010**: Development and initial release of UVM
- **2011**: Release of UVM 1.0
- **2013**: Release of UVM 1.1d with bug fixes
- **2014**: Release of UVM 2.0 with new features
- **2017**: Adoption by IEEE as IEEE 1800.2-2017 standard
- **2020-Present**: IEEE 1800.2-2020

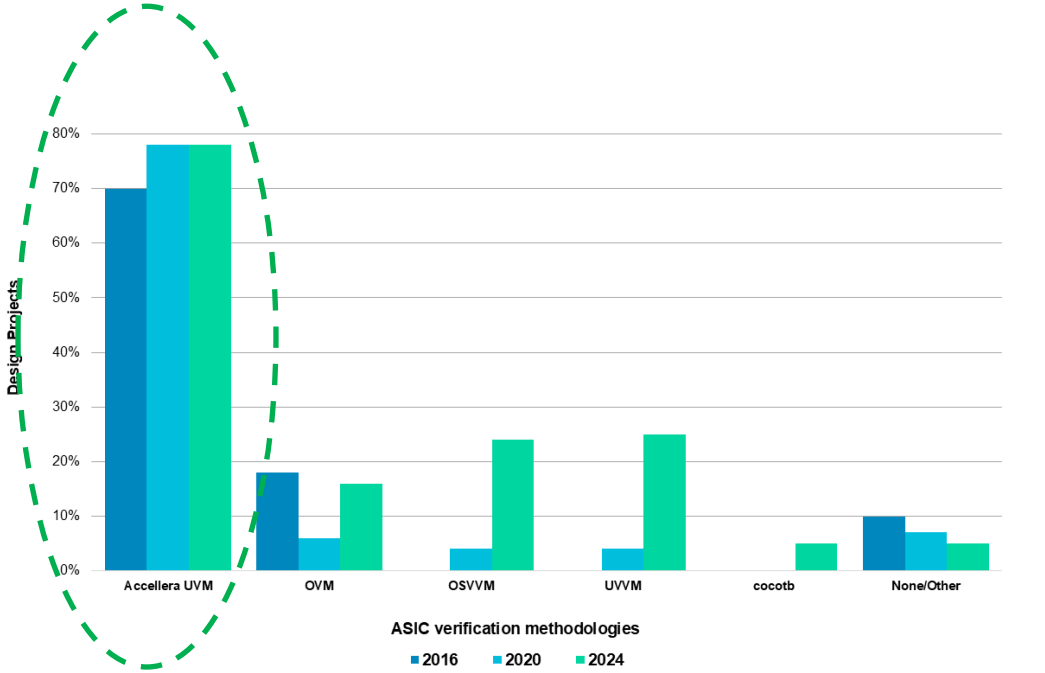



Source: Intel Alumni, UVM history

# Introduction to UVM – TB Methodology Industry Trend




Source: Wilson Research Group and Siemens EDA




Source: Wilson Research Group and Siemens EDA

# Introduction to UVM – UVM Ported to Other Languages

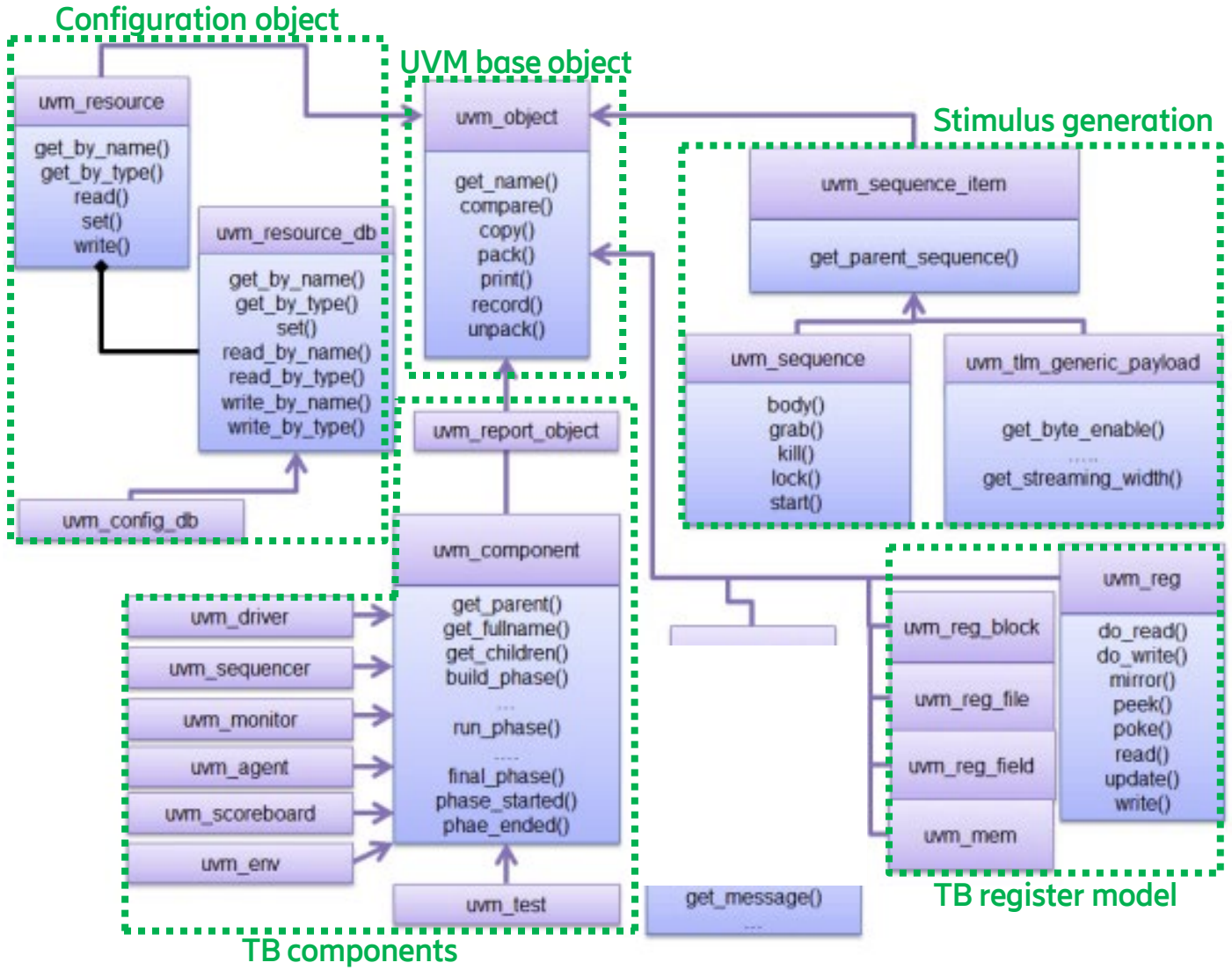
| | |
|---|---|
| **Specman e – eRM (was foundation to UVM)** | • Specman e is a hardware verification language (HVL)<br>• Support the UVM concept |
| **SystemC – UVM** | • C++ based language<br>• Some limitation in the SystemC language like constraints random |
| **Python – Cocotb (Coroutine-based co-simulation TB)** | • Python Cocotb is library with similar UVM capabilities |
| **D Programming Language (Dlang)– E-UVM** | • Dlang is a C-like syntax language that compiles to native code and includes features from SystemVerilog |

# Coffee Break


UVM Concept

# UVM Concept – Class Hierarchy

- **Most used UVM class library**
  - UVM base object
  - Configuration object
  - TB components
  - TB Register model
  - Stimulus generation



# UVM Concept – Execution phases

UVM TB executes the simulation in 3 different main phases

**Build/connect phases (static creation)**

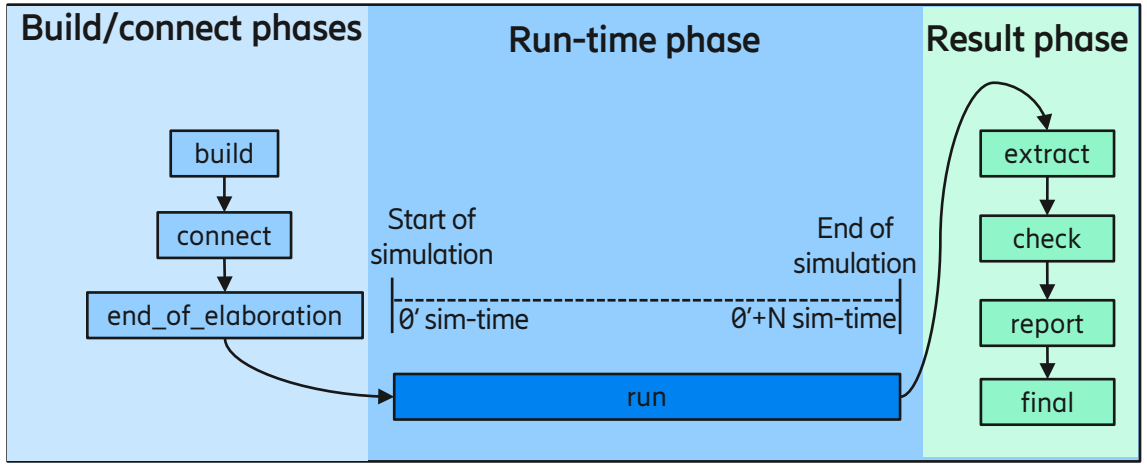
- Used to construct the TB components, connect components and prepare TB before the simulation starts

**Run-time phase (dynamic creation)**

- Start simulate the test that generates the stimulus, run, monitor and check the results

**Result phase**

- Used for extract, check final information from the components and generate result




UVM execution phases

# UVM Concept – Universal Verification Component (uVC)

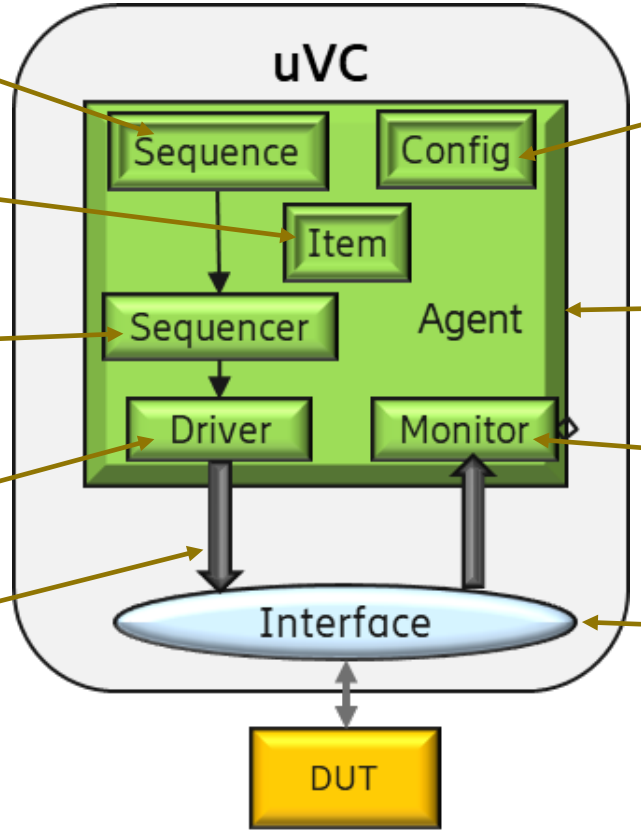
Generates test stimulus by randomizes the sequence item

Consists of data fields required for generating the stimulus

Sequencer controls the flow of sequence requests to the driver

Drive the DUT signals based on the data fields in the sequence item

Pin signal level




Configuration to configure the agent

Agent component encapsulates the reusable uVC in a single environment

Captures DUT signals and translate them into a sequence item format

Connects DUT signals to the agent's driver and monitors via virtual interfaces

Dynamic UVM object
Static UVM component
System Verilog interface
Device Under test

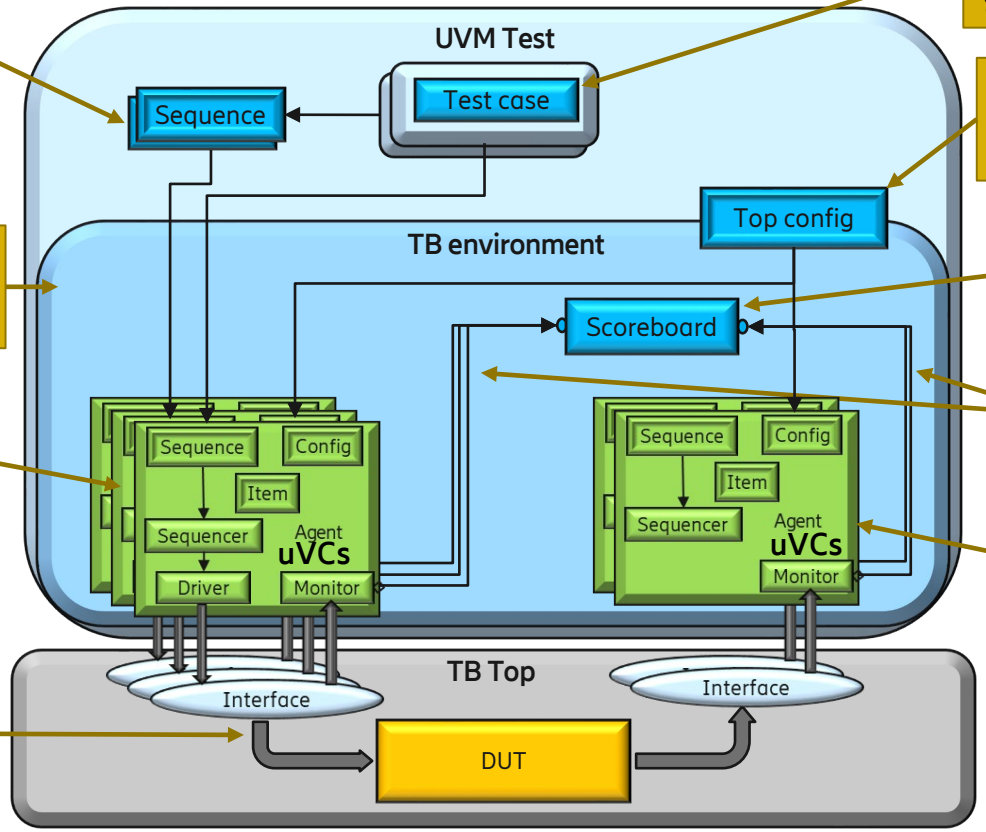
# UVM Concept – UVM Test and Environment

Additional sequences can generate stimulus to the sequencer

The TB environment instantiates all components in the environment

A uVC support a group of signals working together (e.g. bus)

TB top connects DUT signals to interfaces




Basic UVM architecture

A test case creates the TB environment and generates the test stimulus

The environment uses the top configuration to configure the TB environment and uVCs

The scoreboard contains checkers and verifies the functionality of the DUT

Transaction Level-Modeling (TLM) connection

uVC configured as a passive without driving the signals to DUT output signals

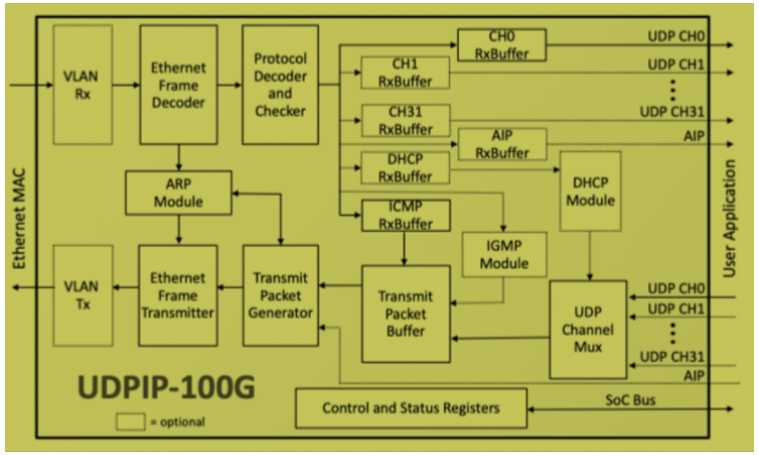
Dynamic UVM object
Static UVM component
System Verilog interface
Device Under test

# UVM TB Reusability and Scalability

# UVM TB Reusability and Scalability – IP Blocks Interconnections in an ASIC

## An ASIC contains many IP blocks

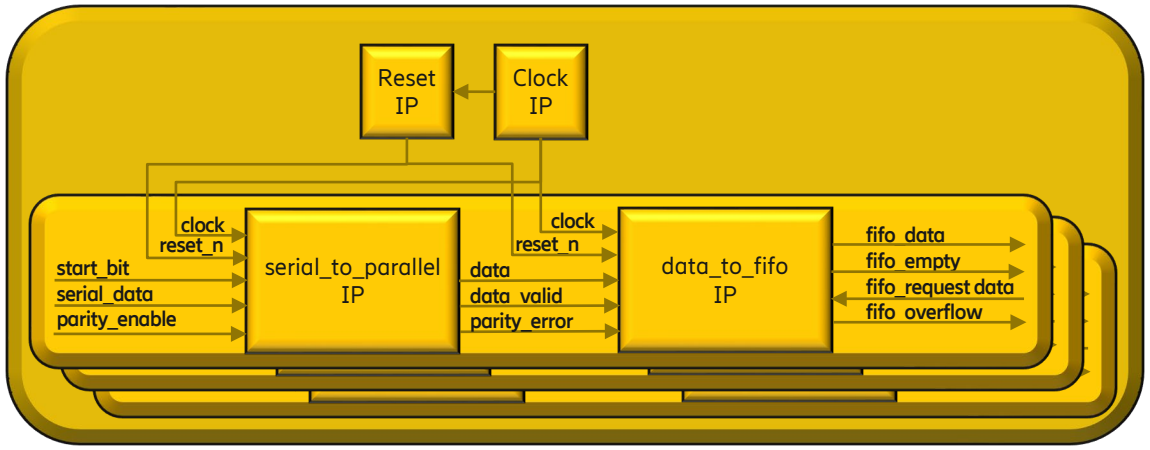
- IP blocks are connected to each other to implement the functionality




Source: www.cast-inc.com

## The same IPs may be instantiated multiple times

- A hierarchy of interconnected subcomponent can be instantiated multiple times




Multiple instances of subcomponent with serial-to-parallel and data-to-fifo IP blocks

# UVM TB Reusability and Scalability – TB for serial-to-parallel Design

**The base test creates the TB environment using the top configuration and runs a basic test**

- A test can change the default top configuration and overwrite the default base test

**DUT input signals connected to clock, reset and serial data uVCs**

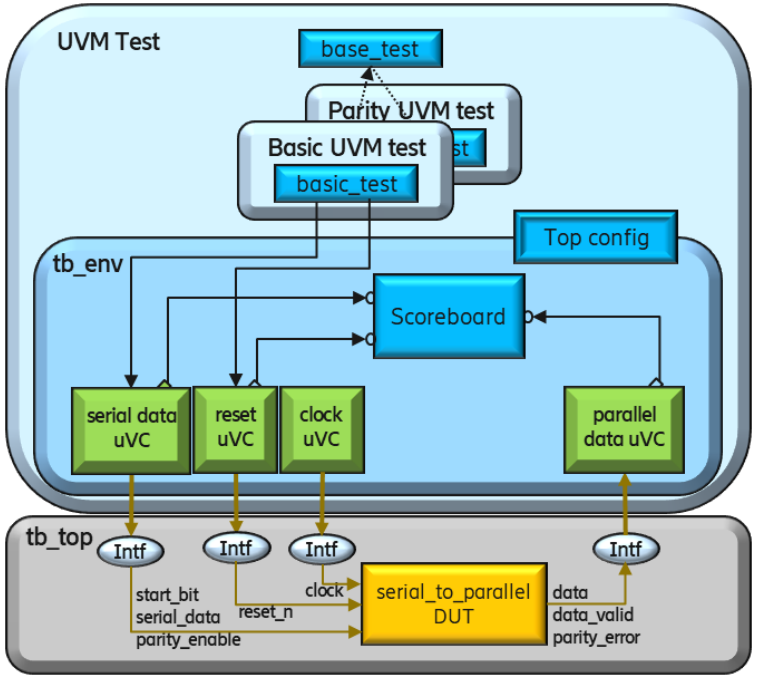
- Drives the stimulus to the DUT
- Reset and serial data monitors send transaction to the scoreboard

**Parallel data uVC monitor the DUT output signals**

- The uVC is set in passive mode means the driver is not created
- The monitor sends output transactions to the scoreboard

**The scoreboard verifies design correctness by comparing actual and expected outputs**

- The scoreboard also includes coverage groups to collect functional coverage

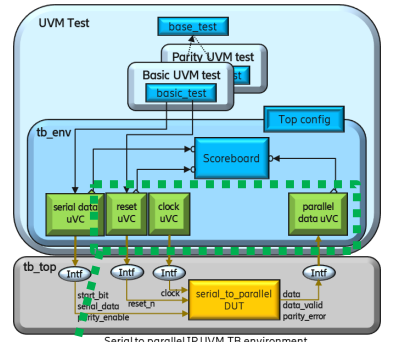



Serial to parallel IP UVM TB environment

serial-to-parallel design receives a serial data stream on a single wire and converts it into a parallel 8 bits data bus.

- It generates a data valid signal when all 8 bits data are received
- It uses a start bit to synchronize the data input stream
- When parity is enabled then an extra serial bit must be sent on the serial stream, and it checks parity and data for even parity

# UVM TB Reusability and Scalability – TB for data-to-fifo Design



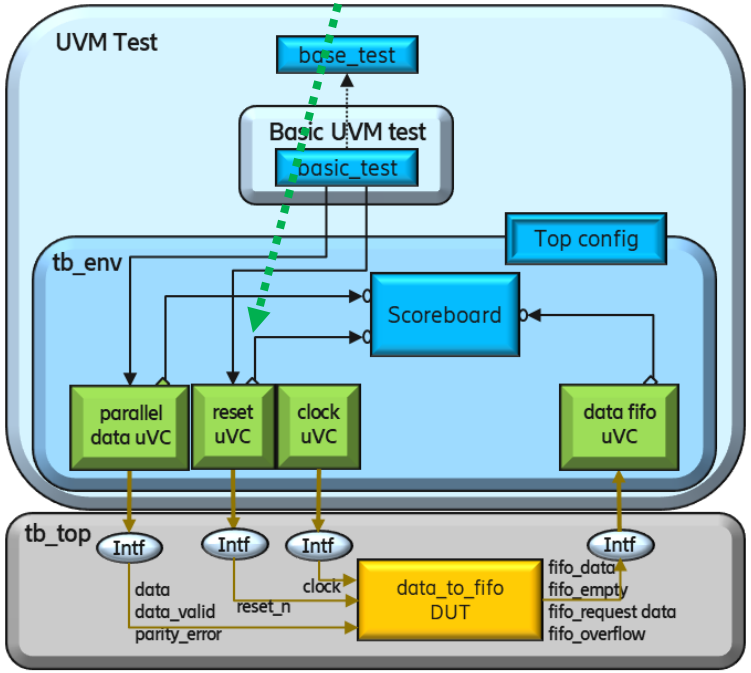
Serial to parallel IP UVM TB environment

## Same concept as UVM TB for serial-to-parallel

- UVC test extends from a base test
- uVCs are instantiated and connected to DUT input/output ports
- uVCs monitors are connected to the scoreboard
- The scoreboard checks expected functionality and includes functional coverage

## Reused clock, reset and parallel data uVCs

- Are all reused directly from serial-to-parallel TB



Data to FIFO IP UVM TB environment

data-to-fifo implements a FIFO buffer with a depth of 4 bytes
- It can store up to 4 bytes of data
- The FIFO has signals for overflow, empty and data availability

**Reusing uVCs between IP TBs increases verification quality and reduces maintenance work**

# UVM TB Reusability and Scalability – Subsystem TB with Vertical Reuse

## Reuse environments from IP UVM TBs to subsystem level

- Reused serial-to-parallel UVM TB environment
- Reused data-to-fifo UVM TB environment

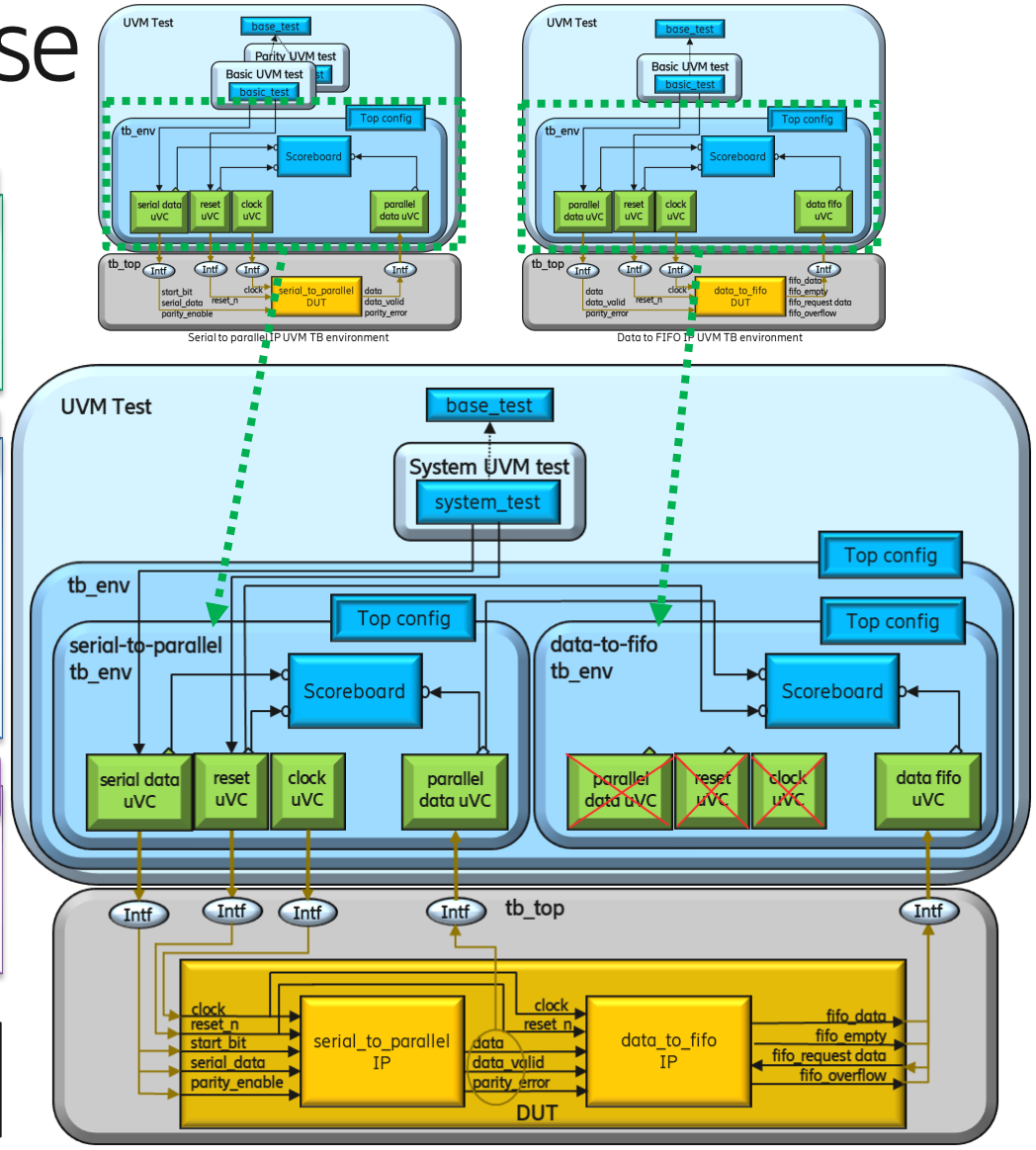
## Adjust the top configurations from reusable environments

- Update configuration to disable creation on redundant uVCs
- Instead, create uVC monitor TLM connection from the other uVC

## Monitoring of signals between IPs in the DUT

- UVC interface signals are connected within the DUT between the IPs

Object-oriented programming simplifies reuse across various verification levels




Subsystem UVM TB with serial to parallel and data to FIFO IP blocks

# UVM TB Reusability and Scalability – Subsystem TB Reusability and Scalability

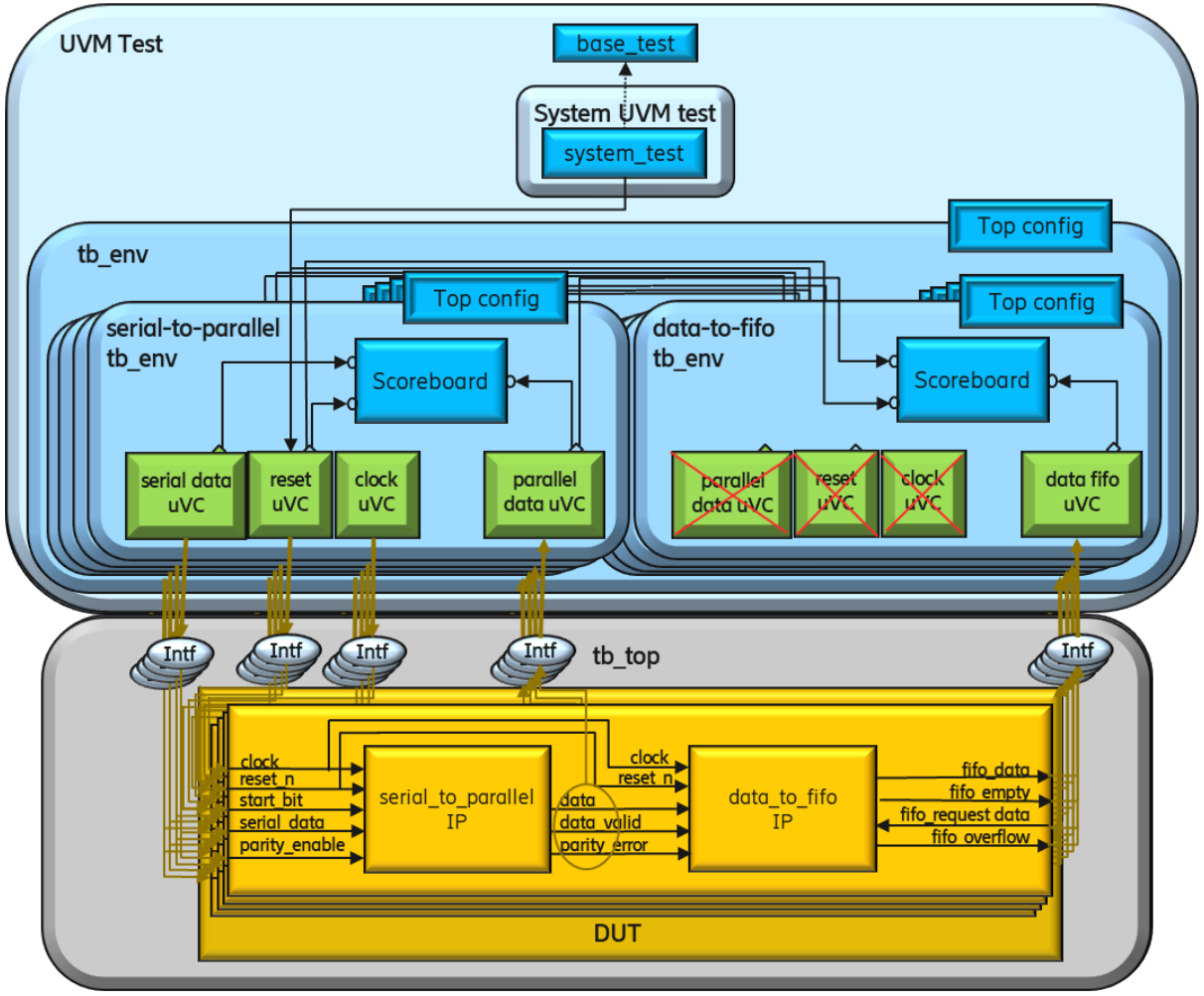
**A DUT may include multiple instances of the same RTL components**

- The TB can easily be scaled to support the verification of the DUT include multiple instances of the DUT interface

**UVM TB supports multiple instantiations of components**

- Multiple instances of a hierarchy across various environments
- Multi instantiation of uVCs

**UVM support an easy way to reuse and scale the TB**




Subsystem UVM testbench with multiple instances of subcomponents featuring serial-to-parallel and data-to-FIFO IP blocks

# UVM Key Takeaways

# UVM Key Takeaways – Advantages

| | |
|---|---|
| **Modularity and reusability** | • The modular nature of UVM facilitates the creation of reusable verification components, reducing development time and improving efficiency |
| **Separation of tests from TBs** | • Separation simplifies test management and enhances the flexibility of the verification environment |
| **Sequence-based stimulus generation** | • Sequence-based stimulus approach enables the creation of detailed and realistic test scenarios, improving verification quality |
| **TB infrastructure creation** | • Dynamic creation and configuration of TB components provide high adaptability and flexibility |
| **Transaction based stimulus** | • Higher level of stimulus abstraction instead of controlling individual signals |

# UVM Key Takeaways – Disadvantages

**Steep learning curve**

- The complexity and extensive feature set of UVM can present a significant learning curve for new users
- Mastering UVM requires a solid understanding of its concepts and methodologies, which can be time-consuming

**Complexity due to object-oriented programming**

- UVM's reliance on object-oriented programming (OOP) adds to the complexity of its class libraries
- This complexity can be challenging for verification engineers who are not well-versed in OOP principles

# UVM Key Takeaways – Engineering Framework



**UVM TB infrastructure**

- Architect and design the TB structure to ensure comprehensive testing of the DUT
- Implement the TB environment with all static components including uVC instantiation and configuration
- Implement the TB top with DUT connection to the uVC interfaces
- Develop uVC's if they are not already available

**UVM test writer**

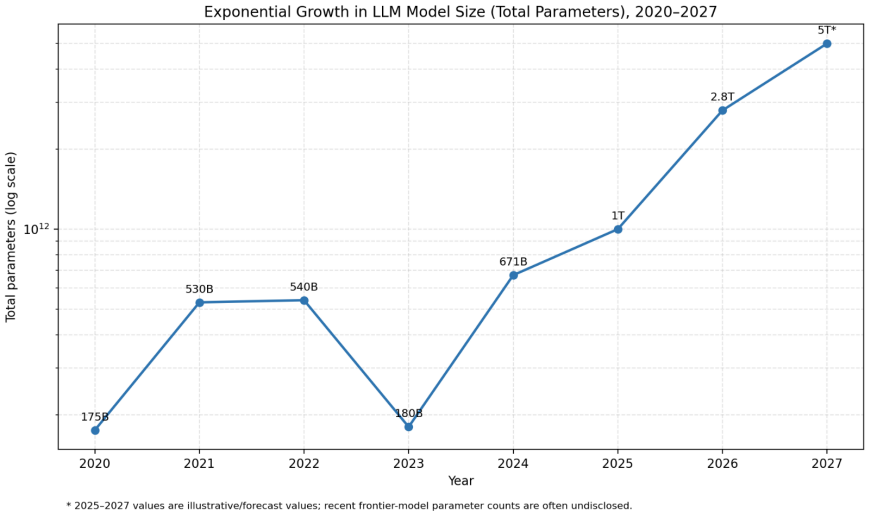
- Implement UVM test cases and sequences to generate the stimulus to the DUT
- Update scoreboard with checkers and/or implement assertion checker
- Add functional cover groups in the TB


How AI Impacts the Way We Work

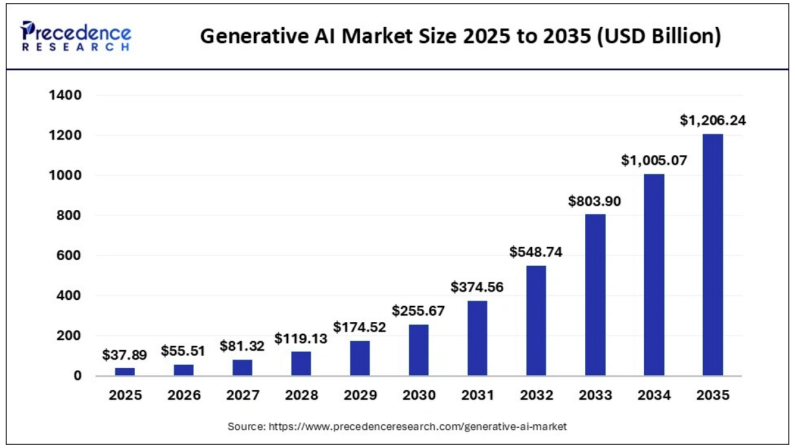
# How AI Impacts the Way We Work

| | |
|---|---|
| LLM Assistant | • Answer questions, explain, summarize, write, code, and analyze |
| RAG (Retrieval-Augmented Generation) | • Retrieve and use company- or domain-specific knowledge |
| Agent | • Plan and execute multi-step tasks using tools and feedback |
| Agentic AI | • Plan, use tools, and take action to complete multi-step tasks |
| AI Skills Library | • Reuse prompts, knowledge, tools, workflows, scripts, and best practices |

AI automates tasks, expands knowledge, and will become essential to use




**Exponential Growth in LLM Model Size**




**The generative AI market has already taken off**


Summary

# Summary



- As designs become more complex and larger, the verification challenge increases
- Creating a traditional SV TB becomes challenging when IP design is updated
- UVM standardizes TB framework using object-oriented methods for flexibility, reusability and scalability
- The UVM concept is based on well-defined components that are extended from UVM base classes
- UVM provides an easy way to both reuse UVM environments and scale the TB
- The UVM engineers' work can be divided in developing TB infrastructure and writing tests
- AI automates tasks, expands knowledge, and will become essential to use

# Q & A

Email: ole.kristoffersen@ericsson.com

# Links

- Easier UVM (doulos.com)
- uvm_guide.book (accellera.org)
- UVM Introduction (verificationguide.com)
- Universal Verification Methodology (learnuvmverification.com)
- UVM API 1.2 (verificationacademy.com)

